

14^강

Java프로그래밍

JDBC 프로그래밍 (교재 13장)

컴퓨터과학과 김희천 교수

- 1 JDBC와 MariaDB
- 2 MariaDB 서버에 연결하기
- 3 MariaDB 연동 프로그래밍
- 4 DatabaseMetaData 객체,
PreparedStatement 객체,
ResultSetMetaData 객체



01

JDBC와 MariaDB

1. 데이터베이스 기초

- 데이터베이스

- 데이터를 모아놓은 집합 장소

- DBMS

- 데이터베이스 관리 시스템
- 데이터베이스의 효과적 관리를 위한 응용 소프트웨어
- 관계형 DBMS에서는 데이터를 테이블 형태로 저장함
 - ✓ DBMS는 여러 데이터베이스들로 구성됨
 - ✓ 하나의 데이터베이스는 여러 테이블들로 구성됨
 - ✓ 하나의 테이블은 여러 레코드(행)들로 구성됨
 - ✓ 하나의 레코드는 여러 필드(열)들로 구성됨
 - ✓ 각 필드에 데이터를 저장하며, 레코드를 식별하는 키 필드가 존재함



2. 관계형 데이터베이스(1)

- 테이블

- 관계형 데이터베이스에서 데이터 구조화를 위한 기본 자료 구조
- 테이블은 고유한 이름을 가짐
- 테이블은 행(레코드 또는 엔터티)과 열(속성 또는 필드)로 구성됨

ID	제목	작성자	작성날짜	조회수	내용
1	시작	홍길동	2024/05/11	1252	첫 번째 글의 내용
2	과정	이순신	2024/05/12	197	두 번째 글의 내용
3	마무리	김유신	2024/05/13	94	세 번째 글의 내용

2. 관계형 데이터베이스(2)

- 스키마

- 테이블의 이름과 구조를 정의
- 테이블의 구조란** 테이블을 구성하는 속성들에 대한 정보

필드 이름	데이터 타입	길이
ID	INT	-
제목	VARCHAR	100
작성자	VARCHAR	20
작성날짜	DATE	-
조회수	INT	-
내용	VARCHAR	500

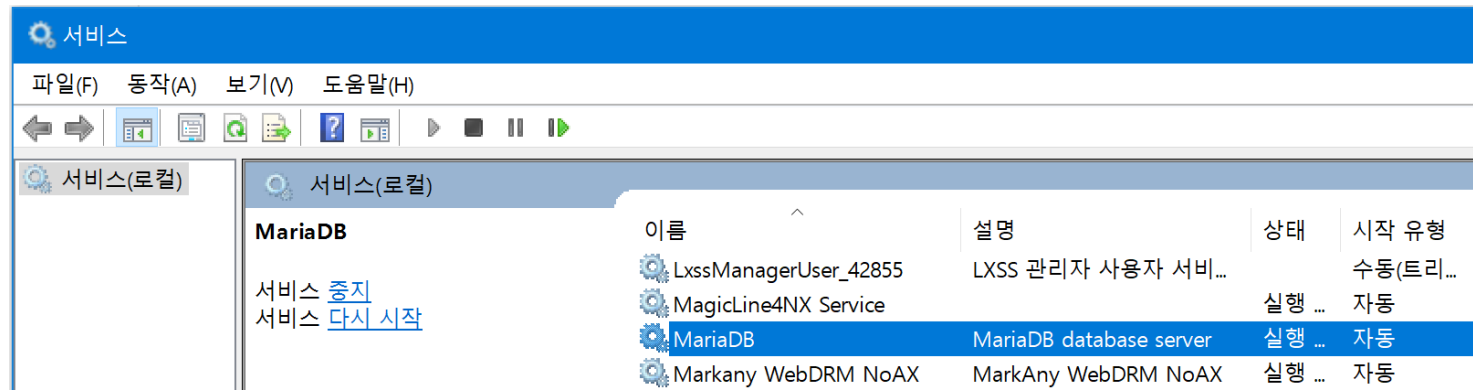
3. DBMS 설치

- MariaDB를 설치하기로 함
 - <https://www.mariadb.org/download/>에 접속
 - **MariaDB Server** 설치 프로그램을 다운로드
 - ✓ mariadb-11.3.2-winx64.msi를 실행시킴
 - ✓ 설치 과정에서 **root 계정의 암호**를 정하고 기억해야 함
 - ✓ 기본적으로 C:\Program Files\MariaDB 11.3\에 설치됨
 - ✓ 기본 포트는 3306
 - <https://dev.mysql.com/doc/index-other.html>에 접속하여 샘플 데이터베이스로 'world database'를 다운로드하여 설치
 - ✓ world-db.zip 파일에서 world.sql을 이용하여 설치



4. MariaDB 사용

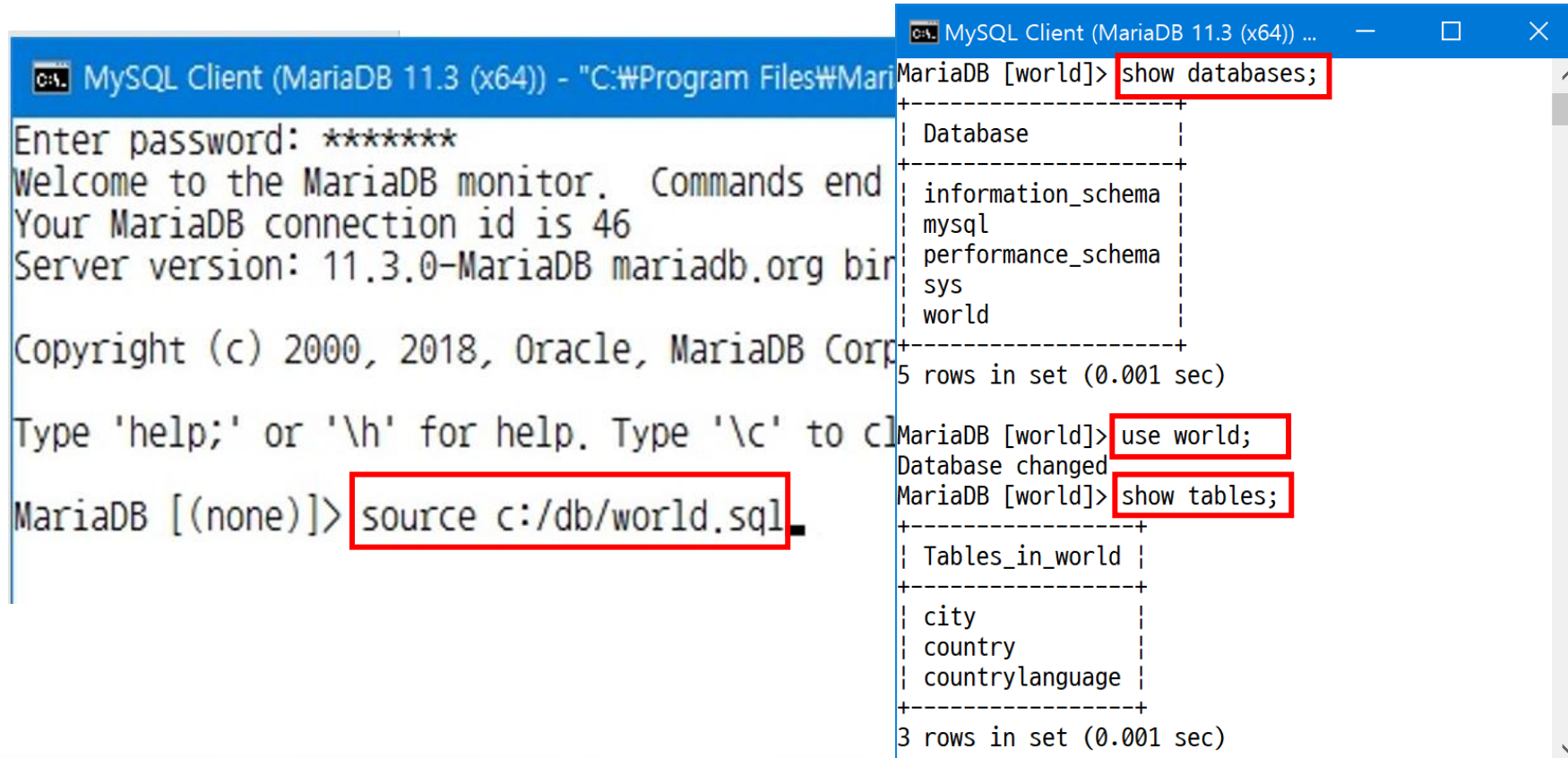
- MariaDB 서버 프로그램이 실행 중인지 확인
 - 데몬을 실행하려면 시작 메뉴의 'Windows 관리 도구'에서 '서비스' 실행
 - ✓ 또는 윈도우 시작 버튼  을 클릭하고, 찾기에서 '서비스'를 타이핑함



- MySQL Client (MariaDB 11.3 (x64)) 프로그램을 실행
 - 시작 메뉴->MariaDB 11.3 (x64)-> MySQL Client (MariaDB 11.3 (x64))
 - 또는 명령 프롬프트 창에서 `mysql -u root -p`를 실행
 - ✓ 먼저 C:\Program Files\MariaDB 11.3\bin 경로를 Path에 추가

5. 샘플 데이터베이스 설치

- <https://dev.mysql.com/doc/index-other.html>에 접속하여 샘플 데이터베이스 'world database'를 다운로드하여 설치
 - world-db.zip 파일에서 world.sql을 이용하여 설치



The screenshot shows two overlapping terminal windows of the MySQL Client (MariaDB 11.3 x64). The background window shows the initial login and the command to source the world.sql file. The foreground window shows the results of the 'show databases;' and 'show tables;' commands.

```
MySQL Client (MariaDB 11.3 (x64)) - "C:\Program Files\MariaDB 11.3\bin"
Enter password: *****
Welcome to the MariaDB monitor.  Commands end with ; or \g
Your MariaDB connection id is 46
Server version: 11.3.0-MariaDB mariadb.org binary distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> source c:/db/world.sql
```

```
MySQL Client (MariaDB 11.3 (x64)) - "C:\Program Files\MariaDB 11.3\bin"
MariaDB [world]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql       |
| performance_schema |
| sys        |
| world       |
+-----+
5 rows in set (0.001 sec)

MariaDB [world]> use world;
Database changed

MariaDB [world]> show tables;
+-----+
| Tables_in_world |
+-----+
| city             |
| country          |
| countrylanguage  |
+-----+
3 rows in set (0.001 sec)
```

The background features a close-up of a laptop keyboard with a futuristic, semi-transparent digital interface overlaid on it. The interface includes various icons like gears, a bar chart, and a search icon, along with some text elements. The overall color scheme is dark blue and teal, with a pinkish-purple gradient at the bottom.

02

MariaDB 서버에 연결하기

1. JDBC

- **JDBC**(Java DataBase Connectivity) **API**
 - Java 프로그램에서 관계형 DBMS와 연동하여 데이터를 시용하고 관리할 수 있게 하는 Java API 표준 규격
 - ✓ 데이터베이스에 연결하고, 데이터베이스에 대해 질의, 갱신, 삽입, 삭제를 요청하고, 결과를 받기 위한 프로그래밍 방법을 제공
 - ✓ JDK의 일부로 포함되어 있음(java.sql 패키지)



2. MariaDB JDBC 드라이버

- JDBC 드라이버

- JDBC API는 DBMS 제조사가 제공하는 JDBC 드라이버를 통해 구현됨
- 사용하려는 DBMS의 JDBC 드라이버를 다운로드 받아 설치해야 함
 - ✓ JDBC API를 사용한 데이터베이스 연동 프로그램을 실행할 수 있음

- MariaDB JDBC드라이버

- MariaDB용 JDBC API를 구현한 소프트웨어
 - ✓ .jar 파일 형식으로 제공됨
- Java 프로그램에서 MariaDB와 연동하는 경우,
‘Java 8+ Connector’가 필요함
 - ✓ <https://mariadb.com/downloads/connectors/>에 접속
 - ✓ mariadb-java-client-3.3.3.jar 파일을 다운로드함



3. MariaDB 연결 프로그램 만들기(1)

- 이클립스 설정

- 자바 프로젝트 생성

- ✓ 프로젝트 이름을 지정(예: JDBCExample)
- ✓ 'module-info.java' 파일을 만들지 말 것

- 드라이버 파일을 **Classpath**에 추가

- ✓ 프로젝트 설정에서 'Project → Properties → Java Build Path → Libraries → Classpath → Add External JARs...'를 이용하여 'mariadb-java-client-3.3.3.jar'를 추가함
- ✓ 또는 마우스 오른 버튼으로 'Build Path → Configure Build Path...' → 'Java Build Path → Libraries → Classpath → Add External JARs...'를 이용

3. MariaDB 연결 프로그램 만들기(2)

- [소스 13-1] MariaDB 접속 프로그램 예

```
String url = "jdbc:mariadb://localhost:3306/world";
String user = "root";
String pass = "mariadb"; //root 계정 비밀번호
try(Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM city limit 0, 50")) {
    System.out.println(String.format("%-10s", "ID") +
String.format("%-35s", "Name") +
String.format("%-15s", "CountryCode") +
String.format("%-20s", "District") + String.format("%-11s", "Population"));
    while(rs.next()) {
        System.out.print(String.format("%-10d", rs.getInt("ID")));
        System.out.print(String.format("%-35s", rs.getString("Name")));
        System.out.print(String.format("%-15s", rs.getString("CountryCode")));
        System.out.print(String.format("%-20s", rs.getString("District")));
        System.out.println(String.format("%-11d", rs.getInt("Population")));
    }
}
```

ID	Name	CountryCode	District	Population
1	Kabul	AFG	Kabul	1780000
2	Qandahar	AFG	Qandahar	237500
3	Herat	AFG	Herat	186800

The background features a close-up, low-angle shot of a laptop keyboard. Overlaid on the image are several semi-transparent, futuristic digital elements. On the right side, there's a large, tilted rectangular panel displaying various icons and data-like patterns. To the left of this panel, a series of vertical bars of increasing height, resembling a bar chart, are visible. In the top left corner, there are faint gear icons. The overall color palette is dominated by dark blues, teals, and greys, with a soft pinkish-purple gradient at the very bottom.

03

MariaDB 연동 프로그래밍

1. MariaDB 연동 프로그래밍 순서(1)

- JDBC 패키지를 import
 - import `java.sql.*`;
- JDBC 드라이버를 동적으로 로드
 - `Class.forName("org.mariadb.jdbc.Driver");` 호출
 - 최신 JDBC 버전에서는 생략해도 됨
- MariaDB 서버 프로그램과 연결 설정
 - `DriverManager.getConnection(url, user, pass)` 호출



1. MariaDB 연동 프로그래밍 순서(2)

- SQL 구문의 실행과 결과 처리
 - Connection, Statement, ResultSet 객체 사용
- 연결 해제
 - 사용 중인 데이터베이스 자원을 반납
 - ✓ finally 블록에서 `rs.close( ); stmt.close( ); conn.close( );` 실행
 - ✓ **try-with-resource** 구문을 사용하면 이 부분이 자동 처리됨



2. MariaDB 서버에 연결하기

- 방법

- `DriverManager.getConnection(url, user, pass)` 메서드는 URL, 사용자 아이디, 비밀번호를 이용하여 MariaDB 서버에 접속을 시도
- 성공하면 `Connection` 유형의 객체를 리턴

```
String url = "jdbc:mariadb://localhost:3306/world";
String user = "root";
String pass = "mariadb";
try(Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM city limit 0, 50")) {
    ... ..
}
```

3. Statement 객체

- SQL 구문을 실행하고 결과를 반환해 주는 객체
- Connection 객체의 `createStatement()` 메소드를 통해 생성됨

- 주요 메소드

메소드	설명
boolean <code>execute</code> (String sql)	SQL 구문을 실행하며, select 구문을 실행하는 경우 true를 리턴하고 이어서 <code>getResultSet()</code> 을 호출할 수 있음. update/insert/delete 구문의 경우 false를 리턴하며 <code>getUpdateCount()</code> 를 호출할 수 있음
ResultSet <code>getResultSet()</code>	SQL 구문(select 구문)을 실행한 결과를 리턴
int <code>getUpdateCount()</code>	SQL 구문(select 구문 제외)의 실행으로 영향을 받은 행의 개수를 리턴
ResultSet <code>executeQuery</code> (String sql)	select 구문을 실행할 때 사용되며, 실행 결과를 나타내는 테이블인 ResultSet 객체를 리턴
int <code>executeUpdate</code> (String sql)	update, insert, delete 구문을 실행할 때 사용하며, 영향 받은 행의 개수를 리턴

4. Statement 객체로 select 구문 실행하기

- Statement 객체를 생성하고 `executeQuery()` 또는 `execute()`를 호출
- 쿼리 결과를 읽을 때는 `ResultSet` 객체에서 읽음

```
try(Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM city limit 0, 50")) {
    while(rs.next()) {
        System.out.print(String.format("%-10d", rs.getInt("ID")));
        System.out.print(String.format("%-35s", rs.getString("Name")));
    }
}
```

```
try(Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement()) {
    ResultSet rs = null;
    if(stmt.execute("SELECT * FROM city limit 0, 50")) rs = stmt.getResultSet();
    while(rs.next()) {
        ...
    }
}
```


5. ResultSet 객체

- **select 구문의 실행 결과를 나타내는 테이블**
 - Statement 객체의 `getResultSet()`, `executeQuery()` 메소드가 리턴하는 객체
 - 테이블에서 한 행을 가리키는 커서를 가짐



Title	author	price
데이터베이스	정재헌	200
C 프로그래밍	홍길동	300

- **select 구문을 실행하여 ResultSet 객체가 생성되면 커서가 만들어지고, select 구문의 실행 결과를 가리킴**
 - 커서는 행을 가리키는 포인터, 기본적으로 위에서 아래로 진행
 - 커서의 초기값은 첫 행의 직전 행을 가리킴

6. ResultSet의 메소드

- 주요 메소드

메소드	설명
<code>boolean next()</code>	커서를 다음 행으로 이동시킴
<code>boolean previous()</code>	커서를 이전 행으로 이동시킴
<code>Statement getStatement()</code>	현재 ResultSet을 생성시킨 Statement 객체를 리턴
<code>String getString(int index), String getString(String columnName)</code>	ResultSet 객체에서 해당 열의 문자열을 리턴. 첫 번째 필드의 인덱스는 1
<code>int getInt(int index), int getInt(String columnName)</code>	ResultSet 객체에서 해당 열의 int 값을 리턴

7. Statement 객체로 insert, update, delete 구문 실행하기

- Statement 객체를 생성하고 `executeUpdate()`를 호출
 - ✓ 영향을 받은 행의 개수가 리턴됨

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement()) {
    int resultCount = stmt.executeUpdate("inset into 테이블이름 values ...");
    System.out.println(resultCount + "개의 행이 삽입되었습니다.");
    resultCount = stmt.executeUpdate("update 테이블이름 set ...");
    System.out.println(resultCount + "개의 행이 변경되었습니다.");
    resultCount = stmt.executeUpdate("delete from 테이블이름 where ...");
    System.out.println(resultCount + "개의 행이 삭제되었습니다.");
}
```

```
if(!stmt.execute("delete from 테이블이름 where ..."))
    System.out.println(stmt.getUpdateCount() + " 개의 행 삭제");
```

04

DatabaseMetaData 객체,
PreparedStatement 객체,
ResultSetMetaData 객체

1. DatabaseMetaData 객체

- 드라이버나 데이터베이스의 정보를 제공하는 메소드를 가진 인터페이스
- Connection 객체의 `getMetaData()` 메소드로 객체를 생성

- 주요 메소드

메소드	설명
<code>String getDriverName()</code>	JDBC 드라이버 이름을 리턴
<code>String getURL()</code>	연결에 사용된 DBMS URL을 리턴
<code>String getUsername()</code>	연결에 사용된 데이터베이스 사용자

2. DatabaseMetaData 사용 예

- [소스 13-2] DBMS 정보 알아내기

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:mariadb://localhost:3306/world";
        String user = "root";
        String pass = "mariadb";
        try {
            Class.forName("org.mariadb.jdbc.Driver");
        } catch (Exception ex) {
            System.out.println(ex);
        }
        try (Connection conn = DriverManager.getConnection(url, user, pass)) {
            System.out.println(conn.getMetaData().getDriverName());
            System.out.println(conn.getMetaData().getDriverVersion());
            System.out.println(conn.getMetaData().getURL());
            System.out.println(conn.getMetaData().getUserName());
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }
}
```

MariaDB Connector/J

3.3.3

jdbc:mariadb://localhost/world?user=root&password=***

root

3. DBMS와 Java의 자료형 변환

- DBMS 테이블에서 필드의 자료형과 Java의 자료형, 그리고 JDBC 메소드 간의 관계

DBMS 자료형	Java 자료형	ResultSet 메소드	PreparedStatement 메소드
CHAR	String	getString()	setString()
VARCHAR	String	getString()	setString()
INTEGER	int	getInt()	setInt()
DATE	java.sql.Date	getDate()	setDate()
			

4. PreparedStatement 객체

- **Precompile된 SQL 문을 표현**
 - 객체는 Connection 객체의 `prepareStatement(String sql)` 메소드를 통해 생성됨
 - ✓ 객체를 생성할 때 SQL 구문이 주어짐(Statement 객체의 경우 실행할 때 제공)
- **같은 SQL 문을 여러 번 반복 실행할 때 효율적임**
 - SQL문에 매개 변수(?)를 사용하고, 실행 전에 값을 지정할 수 있음

```
Connection conn = DriverManager.getConnection(url, user, pass);
String query = "SELECT * FROM 테이블이름 WHERE 필드1=? and 필드2> ?";
PreparedStatement ps = conn.prepareStatement(query);
ps.setString(1, "필드값");
ps.setInt(2, 숫자);
ResultSet rs = ps.executeQuery();
```

5. PreparedStatement의 메소드

- 주요 메소드

- SQL 구문을 PreparedStatement 객체가 가지고 있으므로, SQL 구문을 실행하는 메소드에는 인자가 없음

메소드	설명
boolean execute()	객체가 가진 SQL문을 실행함
ResultSet executQuery()	SQL 쿼리문을 실행하고 ResultSet 객체를 리턴
int executeUpdate()	insert, update 또는 delete 문을 실행
void setInt(int parameterIndex, int x)	SQL 구문의 매개변수(?)에 int 값을 지정. SQL 구문에서 첫 번째 ?의 인덱스가 1임
void setString(int parameterIndex, String x)	SQL 구문의 매개변수(?)에 문자열 값을 지정

6. PreparedStatement의 사용 예

- [소스 13-6] PreparedStatement 객체 사용하기

```
String query = "SELECT * FROM city WHERE population > ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = conn.prepareStatement(query);
    ResultSet rs = getPS(ps, 1, 9000000).executeQuery()) {
    System.out.println(String.format("%-10s", "ID") + String.format("%-35s",
"Name")+ String.format("%-15s", "CountryCode") + String.format("%-20s",
"District")+ String.format("%-11s", "Population"));
    while (rs.next()) {
        System.out.print(String.format("%-10d", rs.getInt("ID")));
        System.out.print(String.format("%-35s", rs.getString("Name")));
        System.out.print(String.format("%-15s", rs.getString("CountryCode")));
        System.out.print(String.format("%-20s", rs.getString("District")));
        System.out.println(String.format("%-11d", rs.getInt("Population")));
    }
}
... ..
```

ID	Name	CountryCode	District	Population
206	São Paulo	BRA	São Paulo	9968485
939	Jakarta	IDN	Jakarta Raya	9604900
1024	Mumbai (Bombay)	IND	Maharashtra	10500000

7. ResultSetMetaData 객체

- ResultSet 객체에서 테이블의 이름, 열의 이름과 타입 정보를 얻을 때 사용되는 객체
 - ResultSet의 `getMetaData()` 메소드로 생성함
- 주요 메소드

메소드	설명
<code>String getColumnName(int index)</code>	index 위치의 컬럼 이름을 리턴
<code>int getColumnCount()</code>	ResultSet의 컬럼 개수를 리턴
<code>int getColumnType(int index)</code>	index 위치의 컬럼 자료형을 리턴
<code>String getTableName(int index)</code>	index 위치의 컬럼을 포함하는 테이블의 이름을 리턴

8. ResultSetMetaData 사용 예

● [소스 13-7] ResultSetMetaData 객체

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM city limit 0, 50")) {
    ResultSetMetaData rsmd = rs.getMetaData();
    int colCount = rsmd.getColumnCount();
    for (int i = 1; i < colCount; i++) {
        System.out.print(rsmd.getColumnName(i) + ", ");
    }
    System.out.println();
    while (rs.next()) {
        for (int i = 1; i < colCount; i++) {
            switch (rsmd.getColumnType(i)) {
                case Types.INTEGER:
                    System.out.print(rs.getInt(i) + ", ");
                    break;
                case Types.CHAR:
                    System.out.print(rs.getString(i) + ", ");
                    break;
            }
        }
        System.out.println();
    }
}
```

```
ID, Name, CountryCode, District,
1, Kabul, AFG, Kabul,
2, Qandahar, AFG, Qandahar,
3, Herat, AFG, Herat,
```


다음시간 안내 >>

15 강

라이브러리와 모듈